

SCAM-SHIELD

Real-Time Payment Fraud Detection Platform

AI Hackathon Submission — Confluent AI Day India 2026

Infosys Limited | EAISCFLT Platform Services

Deterministic Flink Detection + PII-Protected Event Bus + AI-Governed Agentic Overlay
Explainable | Replayable | Regulation-Ready

Table of Contents

1 Executive Summary

2 Problem Statement

3 Solution Overview

4 System Architecture

5 Functional Design

5.1 Payment Channels & Event Types

5.2 End-to-End Transaction Flow

5.3 Fraud Scenario Coverage

5.4 Decision Engine & SLA Tiers

5.5 Complaint Correlator

6 Platform Foundation

6.1 Confluent Cloud Kafka

6.2 Apache Flink (Managed)

6.3 Azure Integration

7 Core Components

7.1 Java Event Producer

7.2 Stream Processing Pipeline (Flink)

7.3 MuleHunter.AI — Cross-Bank Mule Scoring

7.4 AML Engine

7.5 Score Fusion & Decision Engine

7.6 AI Governance Layer

7.7 Agentic AI for Fraud Operations

7.8 Fraud Operations Dashboard

8 Design Invariants

9 Compliance & Regulatory Coverage

10 Security Architecture

11 Infrastructure as Code

12 Observability & Audit

13 Implementation Status

Appendix A Kafka Topic Catalogue

Appendix B Enrichment Lookup Store Inventory

Appendix C MuleHunter Feature Set

Appendix D Key Metrics & Scale

1. Executive Summary

scam-shield is a production-grade, real-time payment fraud detection platform built on Confluent Cloud (Apache Kafka + managed Flink) with deep Azure integration. It protects five payment rail families — UPI, NEFT/RTGS, Cards, Mobile/Net Banking, and IMPS — by applying deterministic stream-processing rules, cross-bank mule-account intelligence (MuleHunter.AI), Anti-Money Laundering checks, and a governed Agentic AI overlay that assists fraud analysts without ever making autonomous decisions on regulated outcomes.

The platform's central thesis is that AI and deterministic rules are not alternatives — they are complementary layers at different points in the decision lifecycle. Flink decides; AI explains, learns, and assists.

Platform Highlights at a Glance

27 Kafka topics | 33 Avro schemas | 8 deployable Flink statements | 15 logical jobs
13-node LangGraph agentic graph | 4 MCP servers | 6 Qdrant RAG collections
22 Terraform modules | 4 environments (dev / pre-prod / prod / prod-DR)
Two-tier SLA: P99 < 120 ms (fast path) | P99 < 350 ms (full ensemble)
Regulatory coverage: PMLA, RBI KYC, FATF sanctions, NPCI, RBI DPIP, NCRP

2. Problem Statement

India's real-time payment infrastructure processes billions of transactions annually across multiple rails — UPI (peer-to-peer and merchant), NEFT/RTGS (high-value bank transfers), debit and credit cards, and internet/mobile banking. This volume, combined with the irrevocable, near-instantaneous settlement model, creates a narrow window for fraud detection. Once a payment is settled, victim recovery rates are low.

2.1 The Core Fraud Vectors

- **SIM-Swap Attacks** — Fraudsters take over a victim's mobile number by convincing the telecom provider to issue a new SIM. Within hours of a SIM change, they log in with the victim's credentials and initiate large transfers, exploiting the OTP channel they now control.
- **Application (APP) Fraud** — A fraudster, impersonating a bank or support agent, convinces the victim to initiate a payment themselves or to share a screen-sharing session. The tell-tale pattern: a new beneficiary is added and a high-value payment follows within 30 seconds.
- **Mule Account Networks** — Proceeds from fraud are layered through networks of mule accounts — accounts opened by recruited individuals or taken over via identity fraud. Money flows in (credit burst), then out in rapid sweeps. Detection requires cross-bank intelligence that no single institution holds alone.
- **Account Takeover (ATO)** — Using stolen credentials or social engineering, attackers access an account from unrecognised devices or locations, change security settings, and drain funds.
- **Card Testing & Structuring** — Stolen card numbers are validated by making small transactions. Structuring — splitting large transfers into sub-threshold amounts — is used to evade CTR (Cash Transaction Report) triggers under PMLA.
- **Cross-Rail Pattern Exploitation** — Sophisticated attacks combine signals across rails: a card test on one rail provides confidence before a high-value NEFT transfer on another. Single-rail detectors miss these patterns entirely.

2.2 Why Existing Approaches Fall Short

- Rule-only systems cannot adapt to novel patterns without manual coding cycles.
 - ML-only systems are opaque — unacceptable in a regulated environment where every decision must be explainable and replayable for audit.
 - Siloed detection (per-rail) misses cross-rail attacks.
 - No single bank has the cross-institution visibility needed to detect mule networks without a collaboration framework (RBIH / MuleHunter).
 - Analyst tooling (dashboards, case management) is usually disconnected from the real-time signal, causing delays in investigation.
-

3. Solution Overview

scam-shield addresses all of the above gaps with a layered, event-driven architecture that cleanly separates concerns while maintaining a fully auditable, end-to-end data lineage.

3.1 The Three-Pillar Architecture

- **Pillar 1: Deterministic Detection Core (Confluent Flink)** — Eight deployable Flink SQL statements implement 15 logical detection jobs — ingest filtering, cross-rail enrichment, per-rail rule scoring, complex event pattern matching (CEP), and ensemble score fusion. The final decision (ALLOW / CHALLENGE / BLOCK / HARD_BLOCK) is always produced by deterministic SQL — never by an AI model.
- **Pillar 2: Intelligence Signals (MuleHunter.AI + AML)** — Two independent consumer groups enrich the decision with cross-bank mule-account probability scores (MuleHunter.AI, operated under the RBIH framework) and regulatory AML checks (PMLA sanctions, PEP screening, structuring detection). Both emit scored events back onto the Kafka bus; Flink fuses them into a weighted ensemble.
- **Pillar 3: Governed Agentic AI (Post-Decision Layer)** — After Flink makes a decision, a 13-node LangGraph agentic graph assists fraud analysts: it enriches alert context from vector memory, generates human-readable case narratives via IBM watsonx / Azure OpenAI, surfaces runbook recommendations, and proposes case actions — all subject to Human-in-the-Loop (HITL) approval for high-value or uncertain cases. AI never writes to a decision topic.

3.2 How AI and Rules Coexist

The boundary between the deterministic zone and the AI zone is the topic `fraud.alert.raised` — a locked, append-only Kafka topic produced exclusively by Flink. The AI layer is a consumer of this topic. It can never write upstream of it. This design guarantees that regulated decisions are always reproducible: replaying the same raw events through the same Flink SQL produces the same decision, regardless of AI model changes.

3.3 Layer Responsibility Matrix

Layer	Technology	Responsibility	AI Decision Authority
Java Producer	Java / Avro / HMAC	PII hashing, event generation, fraud scenario injection	None

Kafka Event Bus	Confluent Cloud	Durable ordered delivery, schema governance, lineage	None
Flink (Ingest)	Confluent Flink SQL	Rail filtering, dedup, auth-pairing, validation	None
Flink (Enrich)	Confluent Flink SQL	11-table temporal joins, velocity state computation	None
Flink (Rules)	Confluent Flink SQL	Per-rail deterministic rule scoring	Rule-based only
Flink (CEP)	Confluent Flink SQL	Cross-rail complex event pattern matching	Rule-based only
MuleHunter.AI	Java / Kafka / Redis	Cross-bank mule probability scoring (RBIH)	None (score only)
AML Engine	Java / Kafka	Sanctions, PEP, CTR/STR checks (PMLA)	None (score only)
Flink (Decide)	Confluent Flink SQL	Ensemble fusion, final decision, freeze commands	Deterministic
AI Governance	Python / Azure Container Apps	10-step model gateway: authN, authZ, PII, audit	None
Agentic AI	Python / LangGraph	Case context, narratives, recommendations, HITL	No decision power
Fraud Ops Dashboard	Vue / FastAPI	HITL interface, alert feed, case management	Human analyst

4. System Architecture

4.1 Functional Architecture

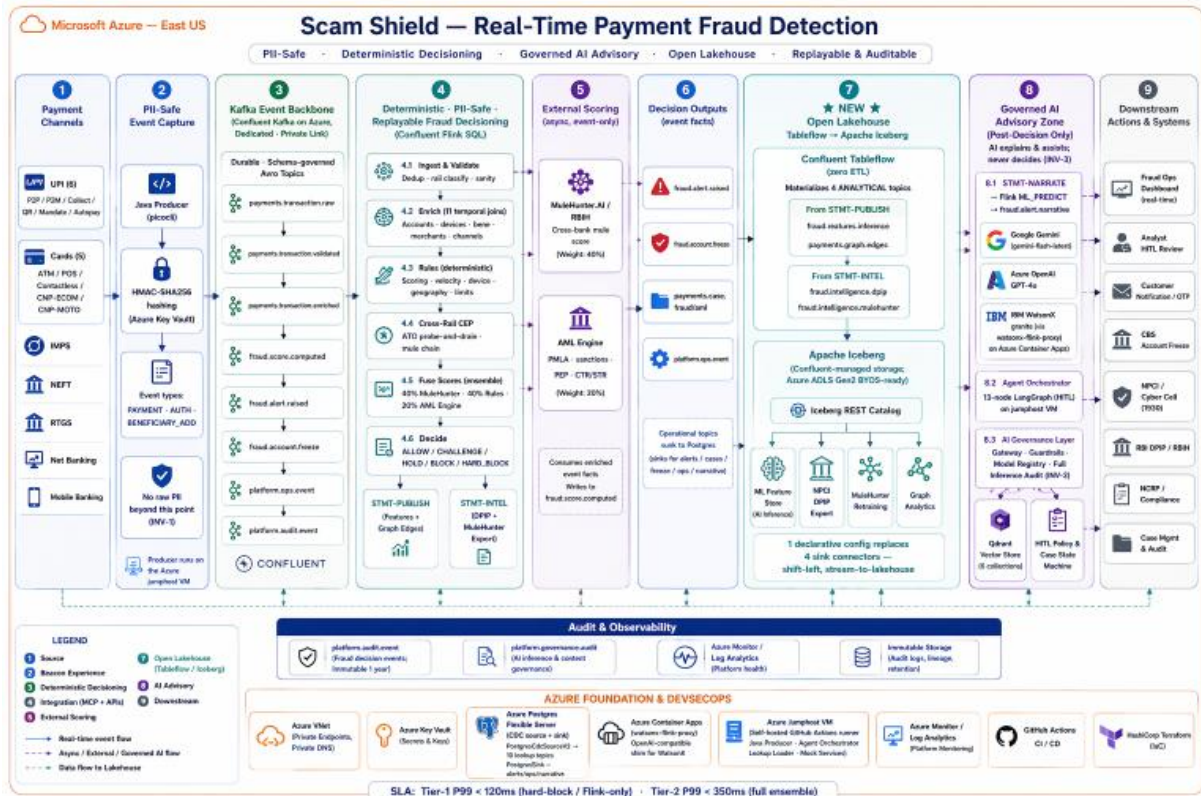


Figure 1 — Functional Architecture: end-to-end fraud detection flow across all platform layers

4.2 Component Interaction Diagram

The diagram below traces the event-driven data flow zone by zone: PII-Protected Production Zone, Kafka Event Bus, Deterministic Detection Zone, Intelligence Signal Zone, and AI Governance Zone.

```
PRODUCTION ZONE
  [Payment Channels: UPI / NEFT / RTGS / Cards / Mobile / IMPS]
  |
  [Java Producer] - HMAC-SHA256 every identifier (INV-1: No raw PII on wire)
  |
KAFKA EVENT BUS (Confluent Cloud - Dedicated, Private Link)
  payments.transaction.raw
  |
DETERMINISTIC DETECTION ZONE (Confluent Flink SQL)
  STMT-INGEST: filter | dedup | validate per rail
  | payments.transaction.validated
  STMT-ENRICH: 11 lookup temporal joins | velocity state
  | payments.transaction.enriched
  |----> STMT-RULES: per-rail rules (SIM swap, velocity, card test ...)
  |                               \ fraud.score.computed (model=FLINK_RULES)
  |----> STMT-CEPXR: cross-rail CEP (impossible travel, ATO ...)
  |                               \ fraud.alert.raised (direct high-severity)
INTELLIGENCE ZONE (independent consumer groups)
  MuleHunter.AI ----+----> fraud.score.computed (model=MULEHUNTER)
  AML Engine  -----+----> fraud.score.computed (model=AML)
  |
  STMT-DECIDE: 350 ms window | ensemble fusion | hard-block first
  |-> fraud.alert.raised (THE LOCKED AGENT CONTRACT)
  |-> fraud.account.freeze -> CBS / NPCI
  |-> platform.audit.event (immutable)
  |
AI GOVERNANCE ZONE
  STMT-NARRATE: ML_PREDICT -> IBM watsonx / Azure OpenAI (governed connection)
  |-> fraud.alert.narrative
  Agentic AI (LangGraph 13-node graph):
  |-> context_enrichment (Qdrant RAG)
  |-> rail-specific analysis nodes
  |-> [HITL interrupt if amount > 5L or confidence < 0.85]
  |-> case_management -> notification -> compliance -> SAR
  |-> platform.agent.action | platform.notification.send
  |
  Fraud Ops Dashboard (Vue SPA :8013/dashboard)
  -> real-time alert feed | HITL queue | case timeline | agent narrative
```

4.3 Key Architectural Decisions

- **Event-Only Integration Boundaries** — Every inter-component boundary uses Kafka topics, not REST API calls. This provides backpressure, replay capability, and an immutable audit trail — critical for regulated environments.
- **Append-Only Topics** — All domain topics are append-only. Mutations (e.g., account state changes) are expressed as new events, not overwrites. This enables deterministic replay.
- **Decoupled SLA Tiers** — The 350 ms ensemble window is chosen to accommodate MuleHunter and AML signal latency without blocking the Flink pipeline. Tier-1 (120 ms) serves hard-block overrides before the ensemble completes.
- **PostgreSQL + LangGraph Checkpointer for HITL** — The agentic graph state is persisted in PostgreSQL using the LangGraph PostgresSaver. A graph execution can be interrupted at any node (e.g., the decision node awaiting analyst approval) and resumed exactly from that point after human input.
- **Qdrant Vector Store for Institutional Memory** — Confirmed fraud case narratives, analyst runbooks, and behavioral profiles are stored in Qdrant. Each new alert retrieves semantically similar historical cases, enabling the agent to surface relevant investigation guidance learned from past events.

5. Functional Design

5.1 Payment Channels & Event Types

The platform covers five payment rail families, expressed as 16 distinct channel enum values. Flink ingest filter jobs route each event to rail-specific sub-pipelines.

Rail Family	Channel Enum Values	Event Types Supported
UPI	UPI_P2P, UPI_P2M, UPI_COLLECT, UPI_AUTOPAY, UPI_INTENT, UPI_QR	PAYMENT, AUTH, BENEFICIARY_ADD
NEFT / RTGS	NEFT, RTGS, IMPS_INTERBANK	PAYMENT, BENEFICIARY_ADD
Cards	CARD_DEBIT, CARD_CREDIT, CARD_PREPAID, CARD_VIRTUAL, CARD_NFC	AUTH, PAYMENT
Mobile / Net Banking	MOBILE_BANKING, NET_BANKING	PAYMENT, AUTH, BENEFICIARY_ADD
IMPS	IMPS_P2P	PAYMENT, BENEFICIARY_ADD

5.2 End-to-End Transaction Flow

The following steps describe the complete lifecycle of a payment event from production to final disposition, including the agentic investigation path for alerted transactions.

Step 1 — Event Production

The Java producer receives a payment event (from a channel integration layer or a synthetic injector for demo). Every personally identifiable field (account numbers, phone numbers, device IDs, IP addresses) is hashed with HMAC-SHA256 using a key from Azure Key Vault before the event is serialised to Avro. No raw PII ever leaves the production zone.

Step 2 — Raw Ingest & Validation (STMT-INGEST)

Flink reads from `payments.transaction.raw` and applies per-rail ingest jobs:

- Sanity checks (amount > 0, mandatory fields present)
- Deduplication (24-hour window keyed on `event_id`)
- Channel classification and rail assignment
- AUTH event pairing with prior PAYMENT events

Rejected events are routed to dead-letter topics for ops remediation.

Step 3 — Enrichment (STMT-ENRICH)

Validated events are enriched with 11 lookup dimensions using temporal joins (`FOR SYSTEM_TIME AS OF`), ensuring time-consistent enrichment for replay fidelity:

- E-1 Account baseline (KYC tier, age, dormancy flag, credit limit)
- E-2 Device baseline (first-seen, trust score, jailbreak flag)
- E-3 Beneficiary baseline (risk score, first-seen, transaction count)

- E-4 Geolocation (country, city, known-risky flag)
- E-5 Merchant profile (MCC, risk tier)
- E-6 Velocity state (debits/hour, credits/hour — Flink keyed state)
- E-7 Mandate registry (auto-pay schedule, max amount)
- E-8 SIM swap log (last SIM change timestamp)
- E-9 Sanctions / PEP list (OFAC, UN, domestic)
- E-10 Complaint history (30-day rolling complaint count)
- E-11 Graph community (network community ID for mule ring detection)

Step 4 — Per-Rail Rule Scoring (STMT-RULES)

Enriched events pass through per-rail deterministic rule sets. Each rule produces a `FraudScoreComputed` event with a score (0–100), `model=FLINK_RULES`, rule code, and contributing features. High-severity single-rule fires are also written directly to `fraud.alert.raised` as Tier-1 fast-path decisions.

Step 5 — Cross-Rail CEP (STMT-CEPXR)

Confluent Flink's `MATCH_RECOGNIZE` operator detects patterns that span multiple events across time: impossible travel (two transactions from geographically distant locations within minutes), SIM swap followed by a new-device transaction, and multi-channel ATO indicators.

Step 6 — Intelligence Signals (MuleHunter.AI & AML)

Two independent consumer groups process `payments.transaction.enriched` in parallel:

- MuleHunter.AI applies 20 features (F1–F20) against a cross-bank mule model, scoring both the sender and beneficiary account for mule probability.
- AML Engine checks against PMLA thresholds ($CTR \geq ₹10$ lakh cumulative), sanctions lists, PEP status, structuring patterns, and grey-list geographies.

Both emit scored events to `fraud.score.computed`.

Step 7 — Ensemble Fusion & Decision (STMT-DECIDE)

Flink opens a 350 ms tumbling window per transaction, collects all score signals, and fuses them:

$$\text{weighted_score} = (\text{FLINK_RULES} \times 0.40) + (\text{MULEHUNTER} \times 0.40) + (\text{AML} \times 0.20)$$

Hard-block rules override the ensemble regardless of score. Missing signals cause weight renormalisation (graceful degradation). The final decision is routed:

- HARD_BLOCK → immediate reject, account freeze
- BLOCK → payment stopped, case opened
- CHALLENGE → OTP challenge or HOLD pending verification
- ALLOW → payment released (audit trail retained)

Step 8 — AI Narrative Overlay (STMT-NARRATE)

After the decision, Flink's `ML_PREDICT` operator calls a governed AI model (IBM watsonx or Azure OpenAI, via a governed Flink connection) to generate a one-paragraph analyst note summarising the evidence trail. This note is written to `fraud.alert.narrative` — strictly downstream of the decision. The narrative cannot modify the decision.

Step 9 — Agentic AI Investigation

For every alert on `fraud.alert.raised`, the Agentic AI runtime (LangGraph graph) orchestrates a 13-step investigation:

- Retrieves semantically similar historical cases from Qdrant
- Routes to a rail-specific analysis node (UPI / Card / Transfer-AML)
- Cross-validates with internal rule tools (Python mirrors of Flink rules)
- Reaches a HITL interrupt for high-value or low-confidence cases
- Opens cases, sends analyst notifications, drafts SAR/NCRP filings
- Captures analyst feedback back to Qdrant (Learning Loop F2)

Step 10 — Downstream Disposition

Approved actions propagate downstream:

- `fraud.account.freeze` → Core Banking System (CBS) + NPCI 1930
- `platform.notification.send` → Customer OTP / advisory SMS/push
- `fraud.intelligence.dpip` → RBI Digital Payment Intelligence Platform
- `fraud.intelligence.ncrp` → National Cyber Reporting Portal filing
- `platform.audit.event` → immutable 30-day (prod) / 10-year (PMLA cases) archive

5.3 Fraud Scenario Coverage

The following fraud scenarios are handled by the platform, each with a specific rule or pattern trigger and a mapped decision outcome.

Scenario	Detection Mechanism	Trigger Condition	Decision
SIM-Swap Attack	STMT-RULES (SIM-SWAP-001)	SIM change < 48 h + high-value payment (> ₹25k)	BLOCK
Application (APP) Fraud	STMT-RULES (APP-001) + STMT-CEPXR	New beneficiary add → payment within 30 s	CHALLENGE
Remote Access Tool (RAT)	STMT-RULES (HARD-BLOCK-001)	Known remote access app on device	HARD_BLOCK
Mule Fan-In / Fan-Out	MuleHunter.AI (F1–F5)	Credit burst + sweep within 1 h across accounts	BLOCK
Account Takeover (ATO)	STMT-CEPXR	New device + SIM swap + high-value	BLOCK/CHALLENGE
Card Testing	STMT-RULES (CARD-TEST-001)	3+ small-amount declines in < 10 min	BLOCK
Structuring (Smurfing)	AML Engine + STMT-RULES	3+ round-amount debits < ₹50k in 4 h	SAR filed
Dormant Account Reactivation	STMT-RULES + MuleHunter F6	No debit activity > 90 days + sudden high-value	CHALLENGE
Cross-Rail Impossible Travel	STMT-CEPXR	Two authorisations in different cities within 5 min	BLOCK
Counterparty Diversity Spike	MuleHunter.AI (F14–F16)	10+ distinct payees in 1 h (mule ring signature)	BLOCK
Sanctions / PEP Hit	AML Engine + Enrichment E-9	Beneficiary on OFAC / UN / domestic list	HARD_BLOCK
CTR Threshold Breach	AML Engine	Cumulative cash in/out ≥ ₹10 lakh in 24 h	CTR filed

5.4 Decision Engine & SLA Tiers

The platform operates two decision tiers with different latency guarantees:

6.1 Confluent Cloud Kafka

The Kafka cluster is the backbone of the platform. All component integrations are event-driven — no component communicates with another via REST API for data exchange.

Attribute	Dev / Pre-Prod	Prod
Cluster Type	Standard (multi-tenant)	Dedicated (single-tenant)
Networking	Public (dev) / VNet peering (pre-prod)	Azure Private Link (no public exposure)
Availability	Single zone	Multi-zone (3 AZs)
Retention (standard topics)	3 days	30 days
Retention (PMLA case topics)	N/A	10 years (archive tier)
Encryption	TLS in-transit	TLS + AES-256 at-rest
Schema Registry	ESSENTIALS tier	ADVANCED tier (private networking)
Schema Compatibility	BACKWARD (all schemas)	BACKWARD (all schemas)
Cluster Linking	N/A	Prod → Prod-DR (unidirectional, hot standby)
Topics	27 (same topology)	27 + lookup compacted topics
Stream Catalog Tags	N/A	PII, GDPR, AUDIT_REQUIRED, GOLD_SLA

6.2 Apache Flink (Confluent Managed)

Confluent's managed Flink service provides the detection core. Flink SQL statements are provisioned as Terraform resources and operated as long-running streaming jobs.

Statement	Logical Jobs	Output Topics
STMT-INGEST	JOB-01 (UPI ingest), JOB-02 (Card ingest), JOB-03 (Transfer ingest)	payments.transaction.validated, dead-letter topics
STMT-ENRICH	JOB-04 (Shared enrichment)	payments.transaction.enriched
STMT-RULES	JOB-05 (UPI rules), JOB-06 (Card rules), JOB-07 (Transfer rules)	fraud.score.computed, fraud.alert.raised (high-severity)
STMT-CEPXR	JOB-08 (Cross-rail CEP)	fraud.alert.raised (pattern alerts)
STMT-DECIDE	JOB-09 (Score fusion), JOB-10 (Notification), JOB-12 (Audit)	fraud.alert.raised, fraud.account.freeze, platform.ops.event, platform.audit.event
STMT-COMPLAINT	JOB-11 (Complaint correlator)	payments.case.fraud (hold upgrade), platform.audit.event
STMT-PUBLISH	JOB-S1 (Feature publisher), JOB-S2 (Graph edges)	fraud.features.inference, payments.graph.edges
STMT-INTEL	JOB-S3 (Intelligence export)	fraud.intelligence.dpip, fraud.intelligence.mulehunter
STMT-NARRATE	AI overlay (post-decision only)	fraud.alert.narrative

Key Flink SQL capabilities used: EXECUTE STATEMENT SET (multi-leg output from a single statement), TUMBLE windowing TVF (350 ms ensemble, 1h intelligence export), MATCH_RECOGNIZE (CEP pattern detection), FOR SYSTEM_TIME AS OF (time-consistent temporal joins), CREATE MODEL + ML_PREDICT (in-stream AI inference).

6.3 Azure Integration

Azure Service	Role
Azure Private Link / Private Endpoints	Zero-public-exposure Kafka and Postgres connectivity

Azure Key Vault	HMAC signing key (producer), API key secrets (rotated via Terraform time_rotating)
Azure Monitor + Log Analytics	Platform health dashboards, KQL alerts
Azure Blob Storage	tfstate backend, schema backups, audit log archive (GRS, soft-delete 30d)
Azure PostgreSQL Flexible Server	Lookup table CDC source, agentic AI checkpointer, dashboard sinks
Private DNS Zones	kafka.<region>.azure.confluent.cloud, flink.<region>.azure.glb.confluent.cloud
Azure Container Apps (prod)	AI Governance Gateway (10-step pipeline)

7. Core Components

7.1 Java Event Producer

The Java producer is the platform's event source, built with Maven, Avro serialisation, and picocli CLI. It is designed both for production channel integration and for hackathon demo via synthetic fraud scenario injection.

Producer Command Tree

Sub-command	Purpose
payments	Generate synthetic PAYMENT, AUTH, and BENEFICIARY_ADD events across all rails
mulehunter-mock	Emit pre-scored mule signals for MuleHunter integration testing
aml-mock	Emit AML-scored events (CTR, STR, PEP hits) for AML engine testing
load-lookups	Seed the 11 enrichment lookup stores in PostgreSQL for demo mode

PII Protection at Source

Before serialisation, every identifier field is replaced with its HMAC-SHA256 digest using a secret from Azure Key Vault. Fields hashed include:

- account_id → account_id_hash
- beneficiary_account_id → beneficiary_account_id_hash
- device_id → device_id_hash
- ip_address → ip_address_hash
- phone_number → phone_number_hash
- merchant_id → merchant_id_hash

The plain-text values are discarded after hashing. They never appear in any Kafka message, Flink state, database column, or log line. This enforces Design Invariant INV-1 at the source before data enters the event bus.

Fraud Scenario Injection (Demo Mode)

Scenario Code	Injection Logic
SIM_SWAP	Emit AUTH with SIM change timestamp < 48 h, then PAYMENT > ₹25k
DORMANT_ACTIVATION	Set account last_debit_ts to > 90 days ago, emit high-value PAYMENT
APP_FRAUD	Emit BENEFICIARY_ADD then PAYMENT for same beneficiary within 30 s

MULE_FANOUT	10 PAYMENT credits in 5 min, followed by 10 PAYMENT debits (sweep)
CARD_TESTING	5 small AUTH events on different merchants within 3 min
STRUCTURING	4 PAYMENT debits of ₹49,500 within 4 hours
HARD_BLOCK_REMOTE	PAYMENT event with known remote-access app device_id_hash

7.2 Stream Processing Pipeline (Flink)

The Flink detection core implements 15 logical fraud detection jobs across 8 deployable SQL statements. Each statement uses EXECUTE STATEMENT SET to produce multiple output topics from a single Flink job graph, minimising resource overhead.

Rule Examples — Per-Rail

- **UPI SIM-SWAP-001:** enriched.sim_change_hours < 48 AND enriched.amount > 25000 → score=85, BLOCK
- **UPI APP-FRAUD-001:** MATCH_RECOGNIZE: event_type='BENEFICIARY_ADD' followed by PAYMENT within 30 s to same beneficiary → score=90, CHALLENGE
- **CARD-TEST-001:** 3+ AUTH events with status=DECLINED and amount < ₹500 within 600 s → score=70, BLOCK
- **TRANSFER-DORMANT-001:** days_since_last_debit > 90 AND amount > 100000 → score=75, CHALLENGE
- **HARD-BLOCK-001:** device_id_hash IN known_rat_devices → score=100, HARD_BLOCK (skip ensemble)
- **VELOCITY-DEBIT-001:** debit_count_1h > 10 AND debit_sum_1h > 500000 → score=80, BLOCK

CEP Patterns — Cross-Rail (MATCH_RECOGNIZE)

- Impossible Travel: AUTH events from cities > 1000 km apart within 5 minutes
- SIM-Swap ATO: SIM change event followed by new-device AUTH within 48 h → high-value payment
- Multi-channel ATO: failed LOGIN on mobile banking, then successful UPI auth from different device

7.3 MuleHunter.AI — Cross-Bank Mule Scoring

MuleHunter.AI is an RBIH-operated cross-bank intelligence service that scores accounts for mule-account probability using signals aggregated across participating banks. It is deployed as an independent consumer on payments.transaction.enriched and produces scored events to fraud.score.computed.

Dual-Account Screening

A key capability of MuleHunter is screening both sides of a transaction — the sender AND the beneficiary — for mule probability. This is critical: a legitimate sender may unknowingly be sending funds to a mule account. The platform surfaces both scores in the ensemble.

Mule Signal Categories (F1–F20)

- **Credit Fan-In Signals (F1–F3)** — Rapid credit inflows from multiple counterparties within a short window — indicative of proceeds consolidation
- **Debit Sweep Signals (F4–F5)** — Large debit sweeps within hours of credit inflows — money layering signature
- **Account Age / Dormancy (F6–F8)** — Recently opened or reactivated accounts receiving unusually large credits

- **New Beneficiary Patterns (F9–F10)** — High-value payments to new beneficiaries within 24 h of account opening
- **Cross-Account Device / IP Fan-Out (F11–F13)** — Same device ID or IP address hash associated with > 5 distinct accounts — classic mule coordinator signature
- **Counterparty Diversity (F14–F16)** — 10+ distinct payees in 1 h — indicative of controlled cash-out nodes
- **Structuring Detection (F17)** — Multiple round-amount transfers staying below CTR thresholds
- **KYC / Vulnerability (F18–F19)** — Low KYC tier, senior or student category accounts targeted for recruitment
- **Velocity Z-Score (F20)** — Transaction velocity normalised against account-type peer group — statistical anomaly detection

MuleHunter state is maintained as 90-day per-account rolling logs and 30-day cross-account device/IP sets (Redis-backed in production, in-memory in demo). A cold-start bootstrap snapshot pre-warms state from 90 days of historical transaction data.

7.4 AML Engine

The AML Engine is an independent consumer service that applies PMLA and regulatory checks to enriched transactions. It does not block payments directly; it scores and files reports.

Check	Regulation	Trigger	Action
CTR Filing	PMLA	Cumulative cash $\geq$ ₹10 lakh / 24 h	File CTR to FIU-IND
STR Filing	PMLA	Suspicious pattern — structuring, split transfers	File STR to FIU-IND
Sanctions Screening	FATF / UN / OFAC	Beneficiary matches list	HARD_BLOCK + SAR
PEP Screening	RBI KYC Master Direction	Politically exposed person or close associate	Enhanced due diligence flag
Grey-List Geography	FATF	Transfer to FATF grey/black-listed jurisdiction	CHALLENGE + flag
Structuring Detection	PMLA §3	3+ sub-threshold transfers within 4 h	SAR + score escalation

7.5 Score Fusion & Decision Engine (STMT-DECIDE)

STMT-DECIDE is the ensemble orchestrator — the single Flink statement that emits the final fraud decision. It never calls an AI model. Its logic is fully deterministic and expressed in Confluent Flink SQL.

Decision Logic (simplified)

1. Collect all `fraud.score.computed` events for the transaction within 350 ms window
2. If any score has `decision_override = HARD_BLOCK` → emit `HARD_BLOCK` immediately (Tier-1)
3. Compute: `weighted_score = (flink_score × 0.40) + (mulehunter_score × 0.40) + (aml_score × 0.20)`
If a signal is absent: renormalise weights proportionally
4. Map `weighted_score` to decision:
 - ≥ 90 → BLOCK
 - 70–89 → CHALLENGE or HOLD (rail-specific)
 - 50–69 → SOFT_FLAG (allow with monitoring)
 - < 50 → ALLOW

5. Emit `fraud.alert.raised` with full evidence envelope (all scores + rule codes + enrichment)
6. If `BLOCK` or `HARD_BLOCK` → emit `fraud.account.freeze`
7. Side-output every decision to `platform.audit.event` (JOB-12 – immutable audit trail)

7.6 AI Governance Layer

The AI Governance Layer is the single control plane through which all model access flows. It enforces Design Invariant INV-2: every model call is authenticated, authorised, rate-limited, PII-scrubbed, and audited.

10-Step Governance Pipeline

- **Step 1 — Authentication** — API key / service principal validated against model registry
- **Step 2 — Authorisation** — Caller role mapped to allowed models (orchestrator, narrator, tools)
- **Step 3 — Rate Limiting** — Per-caller token/minute budget enforced
- **Step 4 — PII Regex Scrub** — Output of Step 1–3 checked for unhashed identifiers before forwarding
- **Step 5 — Prompt Injection Detection** — Heuristic + ML classifier flags adversarial prompt patterns
- **Step 6 — Model Resolution** — Alias resolved to current model version (hot-swappable)
- **Step 7 — Model Call** — Forwarded to Azure OpenAI / IBM watsonx / fallback model
- **Step 8 — Output Safety** — Response checked for policy violations, hallucinated PII
- **Step 9 — Cost + Audit** — Token count, latency, caller, model version written to `platform.governance.audit`
- **Step 10 — Return** — Sanitised response returned to caller

Two AI Access Surfaces

Surface	Used By	Mode	Governance
AI Governance Gateway	Agentic AI runtime (all 13 nodes)	HTTP → Container Apps (prod) / direct (dev)	Full 10-step pipeline
Flink ML_PREDICT (governed connection)	STMT-NARRATE only	In-stream SQL → <code>azure_openai_conn</code> / <code>watsonx_conn</code>	Connection-level: auth, audit, PII scrub

Model Registry

Alias	Purpose	Current Model
<code>agentic-narrator-v1</code>	Standard case narrative generation	<code>ibm/granite-4-h-small</code> (watsonx)
<code>agentic-narrator-v2</code>	Enhanced narrative (high-value cases)	<code>gpt-4o</code> (Azure OpenAI — governed)
<code>agentic-tools</code>	Tool-use capable model for agent graph	<code>ibm/granite-4-h-small</code> (watsonx)
<code>agentic-fallback</code>	Fallback when primary model unavailable	Gemini (direct, via googleai connector)

7.7 Agentic AI for Fraud Operations

The Agentic AI runtime is a LangGraph StateGraph that executes per alert. Its purpose is to assist fraud analysts — not to make or override decisions. It is a post-decision intelligence layer that enriches, explains, recommends, and learns.

13-Node LangGraph Graph

Node	Responsibility
------	----------------

orchestrator_node	Entry point; routes the alert, initialises FraudAgentState
context_enrichment_node	Qdrant RAG: retrieves similar historical cases, runbook excerpts, behavioral profile
upi_fraud_node	UPI-specific cross-validation using Python rule tools
card_fraud_node	Card-specific cross-validation (CNP fraud, card-testing patterns)
transfer_aml_node	Transfer/AML-specific analysis (NEFT, RTGS, cross-border)
pattern_gap_node	Learning Loop F3: if confirmed fraud missed by Flink rules, draft SQL proposal for analyst review
decision_node	HITL interrupt point: pauses if amount > ₹5L or agent confidence < 0.85; resumes on analyst input
case_management_node	Opens/updates fraud case in payments.case.fraud; assigns investigator
notification_node	Sends OTP challenge, customer advisory, or analyst notification via platform.notification.send
compliance_node	Checks whether SAR/NCRP filing is required (amount, pattern type, jurisdiction)
sar_generation_node	Drafts SAR or NCRP filing and emits to fraud.intelligence.ncrp
knowledge_update_node	Learning Loop F1: writes confirmed fraud case summary to Qdrant knowledge collection
output_node	Final state serialisation; emits platform.agent.action events for audit

HITL Policy (Human-in-the-Loop)

The decision_node evaluates a hot-reloadable HITL policy table before emitting actions. Analyst approval is required for:

- Transaction amount > ₹5,00,000 (5 lakh)
- Agent confidence score < 0.85
- Case type = SAR or NCRP filing
- Account freeze of a business account (above SME threshold)
- Novel pattern — no similar historical case found in Qdrant (distance > threshold)

HITL state is persisted in PostgreSQL via LangGraph's PostgresSaver. If the server restarts during a pending HITL, the analyst can resume the exact graph execution from the interrupt node — no alert is lost or re-processed.

Tool Registry (Event-Only Tools)

Tool	Output Topic	Action
freeze_account	fraud.account.freeze	Send freeze command to CBS / NPCI
send_otp_challenge	platform.notification.send	Trigger OTP verification for suspect transaction
notify_customer	platform.notification.send	Send advisory SMS/push to account holder
dismiss_alert	platform.ops.event	Mark alert as false positive with reason
flag_ncrp	fraud.intelligence.ncrp	Initiate NCRP (cybercrime) filing
rbi_dpip_report	fraud.intelligence.dpip	Submit confirmed fraud to RBI DPiP
open_case	payments.case.fraud	Create or update fraud investigation case

No tool writes directly to a database or calls an external API synchronously. All tools emit events onto Kafka; downstream consumers and connectors handle the actual side effects. This preserves the event-sourced audit trail.

Learning Loops

- **F1 — Knowledge Update** — After a case is closed as confirmed fraud, the `knowledge_update_node` serialises the full investigation narrative, rule hits, MuleHunter/AML scores, and analyst notes into a Qdrant document. Future similar alerts retrieve this as RAG context.
- **F2 — HITL Feedback Capture** — When an analyst accepts or overrides an agent recommendation at the `decision_node` HITL step, the feedback (action taken, reasoning) is written back to Qdrant's feedback collection and used to fine-tune future recommendations.
- **F3 — Rule Gap Proposal** — If the `pattern_gap_node` detects that a confirmed fraud was not scored by any Flink rule (relied on MuleHunter or AML alone), it drafts a Flink SQL rule proposal and notifies a senior analyst for review. Approved rules are deployed via the standard Terraform IaC PR workflow (not autonomously).

7.8 Fraud Operations Dashboard

The Fraud Operations Dashboard is a Vue SPA served by a FastAPI backend. It is the primary interface for fraud analysts and the HITL terminal.

Panel	Content
Real-Time Alert Feed	Live stream of <code>fraud.alert.raised</code> events; sortable by severity, rail, amount
HITL Queue	Pending HITL decisions awaiting analyst approval; shows agent recommendation + evidence
Case Timeline	Full event timeline per case: <code>raw</code> → <code>enriched</code> → <code>scored</code> → <code>decided</code> → agent actions
Agent Narrative	Governed AI case narrative from <code>fraud.alert.narrative</code> + agent final summary
Flink Statement Health	Live Flink statement status (<code>RUNNING</code> / <code>DEGRADED</code> / <code>FAILED</code>) + lag metrics
Kafka Topic Metrics	Retained bytes, consumer lag, message rate per topic
Scorecard	Daily / weekly fraud caught, false positive rate, HITL escalation rate

8. Design Invariants

Four design invariants are enforced throughout the platform. Violation of any invariant would compromise either the regulatory compliance posture or the operational integrity of the detection system.

Invariant	Statement	Enforcement Mechanism
INV-1	No raw PII on the wire. Every identifier is HMAC-SHA256 hashed before serialisation.	Java Producer (mandatory pre-serialisation step). Cannot be bypassed without modifying the producer code. Key Vault key is not accessible to any downstream component.
INV-2	All model access is governed. No component may call an AI model directly without passing through the AI Governance gateway or a governed Flink connection.	AI Governance Container Apps in prod. In dev, the governance layer runs as a local FastAPI service. The 10-step pipeline enforces auth, rate-limiting, PII scrub, and audit.
INV-3	Regulated decisions are deterministic and replayable. No AI model participates in producing <code>fraud.alert.raised</code> . AI is post-decision only.	STMT-DECIDE is pure SQL with no <code>ML_PREDICT</code> calls. <code>fraud.alert.raised</code> is produced only by Flink statements, never by the Agentic AI runtime. Topic ACLs (RBAC) enforce write permission.
INV-4	Topic names follow the pattern <code><domain>.<entity>.<event></code> (minimum 3 segments). No topic deviates from this schema.	Terraform regex validation at plan time. Topic creation fails if the name does not match the pattern. Enforced on every environment.

9. Compliance & Regulatory Coverage

Regulation / Framework	Coverage
PMLA (Prevention of Money Laundering Act)	CTR filing ($\geq$ ₹10 lakh/day cumulative), STR filing, 10-year case retention, structuring detection, SAR generation
RBI KYC Master Direction	PEP screening (domestic and international), customer due diligence signals in enrichment layer, complaint handling within mandate window
FATF Guidelines	Sanctions screening (OFAC, UN, domestic), grey-list / black-list geography detection, correspondent bank risk flags
NPCI Rules	Account freeze notification to 1930 helpline, authorization reversal signalling, UPI complaint pathway integration
RBI DPIP (Digital Payment Intelligence Platform)	Confirmed fraud entity reporting via fraud.intelligence.dpip → Tableflow → NPCI DPIP ingest endpoint
RBI Customer Protection Circular	24x7 complaint intake (payments.complaint.intake API), 30-day complaint correlator (STMT-COMPLAINT), preventive HOLD within reporting window
NCRP (National Cyber Reporting Portal)	Cybercrime complaint filing via fraud.intelligence.ncrp → downstream connector

9.1 Data Retention Tiers

Data Class	Retention Period	Storage
Standard fraud alerts	30 days	Kafka topic (prod)
PMLA case data	10 years	Archive tier (Confluent Tiered Storage → Azure Blob)
AI governance audit	7 years	Azure Blob, immutable tier
Platform health metrics	90 days	Azure Monitor / Log Analytics
Terraform state backups	90 days	Azure Blob, versioning enabled, soft-delete 30d

10. Security Architecture

10.1 Network Isolation

- **Private Link (Prod)** — Confluent Dedicated cluster is accessible only via Azure Private Endpoint. No data plane traffic traverses the public internet.
- **Private DNS Zones** — Custom DNS zones resolve Kafka bootstrap and Flink REST FQDNs to Private Endpoint IPs. Split-horizon DNS prevents public resolution.
- **Jump Host** — All administrative operations (Terraform apply, Flink REST calls) route through an Azure VM jump host on the private VNet. Laptop access is via SSH tunnel only.
- **No Public Blob ACLs** — Azure Blob containers for tfstate and audit archive have public access disabled. Access is via Managed Identity from the jump host or CI runner.

10.2 Identity & Access

- One Confluent Service Account per application / Flink statement (least-privilege RBAC)
- Resource-scoped roles: DeveloperRead, DeveloperWrite, ResourceOwner — no wildcard grants
- API keys stored in Azure Key Vault; rotated on a 90-day schedule via Terraform time_rotating
- Dual-slot rotation pattern: new key created, tested, then old key revoked — zero downtime
- GitHub Actions uses short-lived OIDC tokens for Azure SP authentication (no long-lived secrets in CI)

10.3 Data Security

- HMAC-SHA256 PII hashing at producer (INV-1) — raw PII never leaves the production zone
 - Avro schema BACKWARD compatibility enforced — schema evolution cannot break consumers
 - Schema Registry ADVANCED tier (prod) — private-network access only, role-based schema management
 - Confluent Stream Catalog tags (PII, GDPR, AUDIT_REQUIRED) — data lineage and governance metadata
 - All data in transit: TLS 1.2+; at rest: AES-256
 - Soft-delete (30 days) + versioning on all Azure Blob containers — accidental deletion recoverable
-

11. Infrastructure as Code (Terraform)

The entire platform — Confluent Cloud cluster, topics, schemas, Flink statements, connectors, RBAC bindings, Azure networking, Key Vault, monitoring, PostgreSQL, and backup — is managed as Terraform code across two roots and 22 modules.

11.1 Repository Structure

Root	Manages	State File
scam-shield/confluent/	Confluent org, environments, clusters, topics, schemas, Flink compute pool, statements, connectors, RBAC, API keys, cluster linking	confluent/{env}.tfstate
scam-shield/azure-integration/	Azure VNet, Private Link, Private Endpoints, DNS zones, Key Vault, Monitor, PostgreSQL, Blob storage, backup pipelines	azure/{env}.tfstate

11.2 Key Design Principles

- TF-1: Everything is code — root main.tf contains only module calls; no raw resources in root
- TF-2: Config committed, secrets never — tfvars in Git, secrets via TF_VAR_* environment variables
- TF-3: New domain = no Terraform changes — topics auto-discovered from resources/ directory
- TF-4: Least privilege + chargeback intrinsic — one Service Account per application/statement
- TF-5: Topic names validated at plan time — Terraform fails if naming convention is violated

11.3 Environment Matrix

Attribute	dev	pre-prod	prod	prod-dr
Cluster Type	Standard	Standard	Dedicated	Dedicated
Networking	Public	VNet Peering	Private Link	Private Link
Flink CFU	5	10	40	20 (standby)
Availability	Single zone	Single zone	Multi-zone (3 AZ)	Multi-zone (3 AZ)
Schema Registry	ESSENTIALS	ESSENTIALS	ADVANCED	ADVANCED (via exporter)
Cluster Linking	N/A	N/A	Active (prod→dr)	Passive (mirror)
prevent_destroy	false	false	true	true
API Key Rotation	Manual	Manual	90-day auto (time_rotating)	90-day auto

12. Observability & Audit

12.1 Four Distinct Audit Streams

Stream	Topic / Destination	Producer	Content
1 — Platform Health	Azure Monitor / Log Analytics	Confluent Cloud metrics exporter	Cluster throughput, Flink lag, network latency, error rates
2 — RBAC / Config Audit	Confluent Audit Log topic → Blob Sink connector → Azure Blob	Confluent Cloud audit log service	Who changed what: API key creation, RBAC binding changes, schema updates
3 — Fraud Decision Audit	platform.audit.event (Kafka, 30-day retention / 10-year PMLA)	JOB-12 (side-output from STMT-DECIDE)	Every Flink statement action: rule hit, enrichment applied, score computed, decision emitted, notification sent, case opened
4 — AI Inference Audit	platform.governance.audit (Kafka → Blob archive)	AI Governance Gateway (all model calls)	Timestamp, caller SA, model alias+version, input token class, output token count, latency, cost, PII scrub result

12.2 Kafka Topic Metrics (Demo Baseline)

Topic	Retained Bytes (Demo Run)
payments.transaction.raw	~92 MB
payments.transaction.validated	~89 MB
payments.transaction.enriched	~71 MB
fraud.score.computed	~178 MB (all scorer outputs combined)
platform.audit.event	~1.09 GB (side-output from all statements)
platform.ops.event	~459 MB

13. Implementation Status

The following matrix summarises the current implementation state of each platform component as of the hackathon submission date. The platform was deployed end-to-end on Confluent Cloud (Dedicated cluster, Azure East US, Private Link) and demonstrated live.

Component	Status	Notes
Terraform — Confluent root (11 modules)	Implemented	All modules applied; prod cluster torn down post-demo (cost control)
Terraform — Azure root (11 modules)	Implemented	Azure VNet, PE, DNS, Key Vault, PostgreSQL — standing
STMT-INGEST	Implemented	3-rail ingest, dedup, validation, dead-letter routing
STMT-ENRICH	Implemented	11-table temporal joins, velocity state computation
STMT-RULES	Implemented	Per-rail rule scoring, high-severity direct alert
STMT-CEPXR	Implemented	Cross-rail CEP (MATCH_RECOGNIZE patterns)
STMT-DECIDE	Implemented	Ensemble fusion, hard-block first, audit side-output
STMT-COMPLAINT	Implemented	30-day complaint correlator, preventive HOLD
STMT-PUBLISH	Implemented	Feature vectors (JOB-S1) + graph edges (JOB-S2)
STMT-INTEL	Implemented	DPIP + MuleHunter retraining feeds (JOB-S3)

STMT-NARRATE	Implemented (Gemini/dev)	Proven on dev Basic cluster with Gemini free tier; prod needs watsonx proxy
Java Producer	Implemented	Polymorphic Avro, HMAC-SHA256, 7 fraud scenarios, REST Proxy fallback
MuleHunter (mock)	Implemented	WeightedLinearMuleModel (20 features); RBIH production model: integration defined
AML Engine (mock)	Implemented	CTR/STR/sanctions/PEP; production AML system: integration contract defined
Agentic AI Runtime	Implemented	13-node LangGraph, HITL, PostgreSQL checkpointer, 4 MCP servers, RAG
AI Governance Gateway	Implemented (dev mode)	Direct Azure OpenAI / watsonx; prod 10-step Container Apps pipeline: designed
Qdrant Vector Store	Implemented	6 collections, text-embedding-ada-002, 1536-dim
Fraud Ops Dashboard	Implemented	Vue SPA at :8013/dashboard; runs locally + VM via SSH tunnel
Learning Loops F1–F3	Implemented	Knowledge update, HITL feedback, rule gap proposal
Learning Loop F4 (rule deployer)	Designed	Requires IaC PR process — intentionally analyst-gated
Schema Registry (33 schemas)	Implemented	BACKWARD compatibility, auto-registered via Terraform
Private Link + Private DNS	Implemented	All three DNS zones applied; Flink REST reachable from jumphost
Cluster Linking (prod→dr)	Designed + Terraform-ready	Not applied (prod cluster torn down)
Tableflow → Apache Iceberg	Designed	Four analytical exhaust topics; Tableflow provisioning deferred to prod rebuild

Appendix A — Kafka Topic Catalogue

Topic	Domain	Producer	Consumer(s)	Key	Purpose
payments.transaction.raw	payments	Java Producer	STMT-INGEST	account_id_hash	All inbound events (PAYMENT/AUTH/BENEFICIARY_ADD)
payments.transaction.validated	payments	STMT-INGEST	STMT-ENRICH	account_id_hash	Filtered, deduped, rail-classified events
payments.transaction.enriched	payments	STMT-ENRICH	STMT-RULES, STMT-PUBLISH, MuleHunter, AML	account_id_hash	11-table enriched + velocity state
fraud.score.computed	fraud	STMT-RULES, MuleHunter, AML	STMT-DECIDE	txn_id	All scoring signals (FLINK_RULES / MULEHUNTER / AML / BEHAVIORAL)
fraud.alert.raised	fraud	STMT-DECIDE, STMT-CEPXR, STMT-COMPLAINT	Agentic AI, STMT-NARRATE, Dashboard	txn_id	Locked agent contract — final decision envelope
fraud.alert.narrative	fraud	STMT-NARRATE	Dashboard, Agentic AI	alert_id	Governed AI analyst note (post-decision only)
fraud.account.freeze	fraud	STMT-DECIDE	CBS, NPCI 1930	account_id_hash	Account / instrument freeze commands
fraud.features.inference	fraud	STMT-PUBLISH (JOB-S1)	AI Inference Service → Tableflow	txn_id	Feature vectors for external ML scoring
fraud.score.behavioral	fraud	AI Inference Service	STMT-DECIDE (optional)	txn_id	ML behavioral scores (async, optional signal)
fraud.intelligence.dpip	fraud	STMT-INTEL (JOB-S3)	NPCI DPIP	entity_id	Regulatory intelligence export
fraud.intelligence.mulehunter	fraud	STMT-INTEL (JOB-S3)	MuleHunter retraining	community_id	Mule candidate feeds for model retraining
fraud.intelligence.ncrp	fraud	Agentic AI	NCRP filer	case_id	Cybercrime / suspicious activity filings
platform.ops.event	platform	STMT-DECIDE, Agentic AI, AML	Dashboard, RBI DPIP	alert_id	All operational outcomes
platform.audit.event	platform	Every STMT-* (JOB-12)	Immutable audit store	event_id	Fraud decision audit trail
platform.governance.audit	platform	AI Governance Gateway	Immutable governance archive	request_id	Model inference audit (every call)
platform.notification.send	platform	Agentic AI	Notification dispatcher	account_id_hash	OTP / customer advisory
platform.agent.action	platform	Agentic AI	Audit,	case_id	Tool actions (freeze,

	m		Dashboard		challenge, flag, dismiss)
payments.complaint.intake	payments	24x7 API	STMT-COMPLAINT (JOB-11)	account_id_hash	Unauthorized transaction complaints (RBI CP)
payments.case.fraud	payments	STMT-DECIDE, STMT-COMPLAINT	Case mgmt, Agentic AI	case_id	Fraud case open + updates
payments.case.aml	payments	STMT-DECIDE, AML Engine	AML team, Agentic AI	case_id	AML case open + updates
payments.graph.edges	payments	STMT-PUBLISH (JOB-S2) → Tableflow	Graph DB	src_node_id	Entity relationship edges (network graph)
payments.transaction.dead-letter.sanity-fail	payments	STMT-INGEST	Ops remediation	txn_id	Rejected events (amount ≤ 0, missing fields)
lookup_account_baseline (compacted)	lookups	Postgres CDC	STMT-ENRICH (E-1)	account_id_hash	KYC tier, age, dormancy, credit limit
lookup_device_baseline (compacted)	lookups	Postgres CDC	STMT-ENRICH (E-2)	device_id_hash	Trust score, first-seen, jailbreak flag
lookup_beneficiary_baseline (compacted)	lookups	Postgres CDC	STMT-ENRICH (E-3)	beneficiary_account_id_hash	Risk score, first-seen, transaction count
lookup_geolocation (compacted)	lookups	Postgres CDC	STMT-ENRICH (E-4)	ip_address_hash	Country, city, risky flag
lookup_sanctions_pep (compacted)	lookups	Postgres CDC / feed	STMT-ENRICH (E-9)	beneficiary_account_id_hash	OFAC, UN, domestic sanctions and PEP lists

Appendix B — Enrichment Lookup Store Inventory

ID	Store Name	Kafka Topic (compacted)	Key	Key Fields Provided to Flink
E-1	Account Baseline	lookup_account_baseline	account_id_hash	kyc_tier, account_age_days, dormancy_flag, credit_limit, is_business
E-2	Device Baseline	lookup_device_baseline	device_id_hash	device_first_seen, device_trust_score, jailbreak_flag, os_version
E-3	Beneficiary Baseline	lookup_beneficiary_baseline	beneficiary_account_id_hash	beneficiary_risk_score, beneficiary_first_seen, beneficiary_txn_count
E-4	Geolocation	lookup_geolocation	ip_address_hash	country_code, city, is_vpn, is_risky_geo, latitude, longitude
E-5	Merchant Profile	lookup_merchant_profile	merchant_id_hash	mcc_code, merchant_risk_tier, merchant_country
E-6	Velocity State	Flink keyed state (internal)	account_id_hash	debit_count_1h, debit_sum_1h, credit_count_1h, credit_sum_1h, distinct_payees_1h
E-7	Mandate Registry	lookup_mandate_registry	mandate_id	mandate_type, max_amount, schedule, issuing_bank
E-8	SIM Swap Log	lookup_sim_swap_log	phone_number_hash	last_sim_change_ts, sim_change_count_30d
E-9	Sanctions / PEP List	lookup_sanctions_pep	beneficiary_account_id_hash	is_sanctioned, is_pep, sanction_list_code, pep_category
E-10	Complaint History	lookup_complaint_history	account_id_hash	complaint_count_30d, last_complaint_ts, open_dispute_flag
E-11	Graph Community	lookup_graph_community	account_id_hash	community_id, community_risk_score, community_size, is_mule_ring

Appendix C — MuleHunter Feature Set (F1–F20)

Feature ID	Name	Description
F1	Credit Inbound Count (1h)	Number of distinct credit transactions received in the last 1 hour
F2	Credit Inbound Sum (1h)	Total credit amount received in the last 1 hour
F3	Credit Source Diversity (1h)	Number of distinct sender account hashes in last 1 hour
F4	Debit Sweep Velocity (1h)	Total debit amount in the 1 hour following a credit burst
F5	Debit-to-Credit Ratio (1h)	Ratio of debit sum to credit sum in rolling 1-hour window
F6	Account Age (days)	Days since account opening — proxy for mule recruitment lag
F7	Dormancy Flag	Boolean: no debit activity in prior 90 days before current event
F8	Days Since Last Debit	Number of days since the last outbound transaction
F9	New Beneficiary High Value	High-value payment to a beneficiary added within 24 hours
F10	New Beneficiary Rate (7d)	Number of new beneficiaries added in the past 7 days
F11	Shared Device Fan-Out	Number of distinct account hashes sharing the same device_id_hash
F12	Shared IP Fan-Out	Number of distinct account hashes sharing the same ip_address_hash
F13	Device IP Combo Reuse	Same device+IP combo observed across > 3 distinct accounts
F14	Payee Diversity (1h)	Number of distinct payee account hashes in the last 1 hour
F15	Payee Diversity (24h)	Number of distinct payee account hashes in the last 24 hours
F16	Cross-Bank Payee Ratio	Fraction of payees at banks other than the account's home bank
F17	Structuring Flag	Boolean: 3+ transfers with amounts just below ₹50,000 in last 4 hours
F18	KYC Tier	Customer KYC level (V1/V2/V3) — lower tier = higher vulnerability
F19	Customer Segment	Account holder segment (senior citizen, student, etc.)
F20	Velocity Z-Score	Standard deviations above peer-group mean for debit velocity

Appendix D — Key Metrics & Scale

Metric	Value
Kafka Topics	27 (+ lookup compacted topics)
Avro Schemas	33 (BACKWARD compatibility enforced)
Flink Statements (deployable)	8 (+ STMT-NARRATE AI overlay)
Logical Flink Jobs	15 (JOB-01 to JOB-12 + JOB-S1/S2/S3)
Agentic AI Graph Nodes	13 (LangGraph StateGraph)
MCP Servers	4 (fraud-scoring, fraud-investigation, aml-triage, fraud-action)
Qdrant Collections	6 (runbooks, behavioral profiles, session embeddings, tx features, KB, tickets)
Terraform Modules	22 (11 Confluent + 11 Azure)
Environments	4 (dev / pre-prod / prod / prod-DR)
Payment Rails	5 (UPI, NEFT/RTGS, Cards, Mobile/Net Banking, IMPS)
Channel Enum Values	16
Enrichment Lookup Stores	11
MuleHunter Features	20 (F1–F20)
Fraud Scenarios Injected (demo)	7 (SIM_SWAP, DORMANT_ACTIVATION, APP_FRAUD, MULE_FANOUT, CARD_TESTING, STRUCTURING, HARD_BLOCK_REMOTE)
Audit Streams	4 (platform health, RBAC/config, fraud decision, AI inference)
Regulatory Frameworks Covered	7 (PMLA, RBI KYC, FATF, NPCI, RBI DPIP, RBI CP Circular, NCRP)
Decision Latency — Tier-1 (fast path)	P99 < 120 ms
Decision Latency — Tier-2 (full ensemble)	P99 < 350 ms
MuleHunter Throughput	P99 < 500 ms; ≥ 500 events/sec per pod
Producer Throughput (demo)	50 txns/sec (configurable)
Java Producer Test Coverage	37 unit tests